A Practical Introduction to LangChain and RAG

A four-page sample document for learning PDF ingestion, chunking, retrieval, and grounded question answering.

Learning objectives

By the end of this sample, a learner should be able to explain the role of a PDF loader, a text splitter, an embedding model, a vector store, a retriever, and a language model in a basic RAG pipeline.

What is Retrieval-Augmented Generation?

Retrieval-Augmented Generation (RAG) is a design pattern that combines information retrieval with text generation. Instead of asking a language model to answer only from its internal learned parameters, an application first retrieves relevant passages from a collection of documents. Those passages are then provided as context to the model.

RAG is useful when answers should be based on a specific collection of documents, such as technical manuals, university policies, product documentation, or internal reports. It does not automatically guarantee correctness: the retrieved passages must be relevant, and the model can still misinterpret them.

Important distinction

RAG does not retrain or fine-tune the language model. It supplies selected external information at answer time. The document collection can be updated without retraining the model, although the indexing pipeline may need to be run again.

1. The Main Stages of a RAG Pipeline

A simple RAG application usually has an indexing stage and a question-answering stage. These stages are related, but they do different jobs.

Indexing stage

1. Load: Read files such as PDFs and extract their text. A loader may also attach metadata, for example the source filename or page number.

2. Split: Divide long text into smaller chunks. Chunk size and overlap influence what information stays together and how much repeated text appears between neighboring chunks.

3. Embed: Convert each chunk into a numerical vector using an embedding model. The vector represents aspects of the text's meaning in a form that supports similarity comparisons.

4. Store: Save vectors together with the corresponding text and metadata in a vector store. The original text is important because the language model needs readable passages, not just vectors.

Question-answering stage

5. Embed the question: Convert the user's query into a vector using a compatible embedding model.

6. Retrieve: Search for chunks whose vectors are close to the query vector according to a chosen similarity or distance measure.

7. Generate: Place the question and retrieved passages into a prompt and ask a language model to produce an answer.

The retrieved text is often called context. A useful prompt instructs the model to answer from that context and to say when the context does not contain enough information.

Practical caution

Retrieval quality depends on document extraction, chunk boundaries, embedding suitability, and search configuration. A fluent answer is not proof that retrieval was correct.

2. Where LangChain Fits

LangChain is a framework for composing applications around language models. Its components provide common interfaces for tasks such as loading documents, splitting text, creating prompts, calling models, and connecting retrieval to generation. It coordinates components; it is not itself the PDF, embedding model, vector database, or language model.

Common components in a beginner PDF-RAG project

Document loader: Turns a source file into one or more Document objects. A Document commonly contains page_content (the extracted text) and metadata (source details).

Text splitter: Creates smaller Document chunks while preserving useful metadata.

Embeddings interface: Defines how text is converted into vectors. The actual embedding computation is performed by a configured embedding model.

Vector store: Offers operations for storing vectors and searching for similar documents. Qdrant is an external vector database; the LangChain integration connects to it.

Retriever: Provides a common way to ask for relevant Documents. It hides some search-specific details behind a retrieval interface.

Prompt template: Builds a prompt from variables such as the user's question and retrieved context.

Chat or text model: Generates the final natural-language answer. A local model server such as Ollama can provide the model.

Typical package roles

langchain-core: Core interfaces and runnable abstractions. **langchain-text-splitters:** text splitting. **langchain-community:** community integrations. Provider-specific packages such as **langchain-ollama** and **langchain-qdrant** connect to particular services.

Package names and recommended imports can change across versions. Use the current official documentation for the chosen version rather than mixing code from older tutorials.

3. Testing a PDF-RAG System

A small RAG project should be evaluated at more than one point. Looking only at the final answer makes it difficult to locate the source of an error.

Useful checks

Document extraction: Confirm that the loader extracted the expected text and that pages are not empty or scrambled.

Chunking: Inspect several chunks. Check whether sentences or definitions are split in unhelpful places and whether overlap creates excessive duplication.

Retrieval: Ask questions whose answers are explicitly present in the sample. Inspect the returned passages and their metadata before involving the language model.

Grounded generation: Compare the answer with the retrieved context. Test a question whose answer is absent and check whether the model acknowledges the limitation.

Example questions for this document

1. Does RAG retrain the language model?
2. What information can a Document object contain?
3. What is the difference between an embedding and a retrieved text chunk?
4. Why should retrieval be tested separately from generation?
5. What should the model do when the retrieved context does not answer the question?

Suggested beginner workflow

First, load the PDF and inspect the returned objects. Second, split the documents and inspect the chunks. Third, create embeddings and store the chunks. Fourth, test retrieval alone. Only after those steps work should you connect the retrieved context to the language model.

This sample is an educational document, not a copy of the official LangChain documentation. For current APIs and exact package instructions, consult the official LangChain Python documentation and integration references.